Manual Oficial de GitFlow y Control de Versiones

Proyecto:	Calculadora de Álgebra Lineal (MTM0120)
Institución:	Universidad Americana (UAM)
Equipo Responsable:	Grupo G-3
Duración del Ciclo:	16 Semanas
Arquitectura:	Backend (Python puro) / Frontend (UI Reactiva)

1. Arquitectura de Ramas

Para garantizar la estabilidad del entorno y evitar conflictos de integración entre la lógica algorítmica y la interfaz de usuario, se establece la siguiente topología estricta de repositorios. **Queda terminantemente prohibido realizar commits directos a ramas protegidas (`main` y `develop`).**

- `main` **(Producción / Cortes Evaluativos):** Rama inmutable que refleja el estado estable y entregable del software. Únicamente recibirá actualizaciones en los tres cortes evaluativos definidos por la UAM.
- `develop` **(Integración Continua):** Rama base para el ciclo de vida diario. Representa la versión más reciente del código integrado y funcional. Todo el código de las nuevas características converge aquí tras su revisión.
- `feature/semana-X-<nombre-modulo>` **(Módulos Efímeros):** Ramas de desarrollo temporal donde se construye la lógica o la UI. Deben tener un ciclo de vida corto (máximo una semana).

  > Ejemplo: `feature/semana-3-gauss-jordan`

- `hotfix/<nombre-falla>` **(Correcciones Críticas):** Ramas de emergencia bifurcadas directamente de `main` o `develop` para solventar caídas del entorno o errores críticos bloqueantes.

2. Flujo Diario de Desarrollo (Protocolo de Terminal)

Todo desarrollador del Grupo G-3 debe ejecutar metódicamente la siguiente secuencia de comandos en Git Bash al iniciar y finalizar su jornada de programación. No se permiten atajos que comprometan la sincronización.

Paso 1: Sincronización del entorno local con la rama de integración

```
git checkout develop
git pull origin develop
```

Paso 2: Aislamiento del nuevo módulo de álgebra o UI

```
git checkout -b feature/semana-<numero>-<nombre-descriptivo>
```

Paso 3: Desarrollo y empaquetado de cambios

```
# Agregar únicamente los archivos modificados pertinentes al módulo
git add <ruta/al/archivo_python_o_ui>

# Registrar el cambio siguiendo la convención estricta
git commit -m "<tipo>(<alcance>): <descripcion>"
```

Paso 4: Publicación en el servidor remoto

```
git push -u origin feature/semana-<numero>-<nombre-descriptivo>
```

3. Estandarización de Historial (Conventional Commits)

Para evitar la ambigüedad en la auditoría del repositorio, todo commit debe adherirse al estándar "Conventional Commits". Cualquier código subido con un mensaje genérico será rechazado en la etapa de revisión.

- `feat` **(Característica):** Implementación de una nueva función algorítmica o componente visual.

  > Ejemplo: `feat(core): implementar eliminación de Gauss-Jordan para matrices NxN`

- `fix` **(Corrección):** Solución a un error de cálculo, lógica o renderizado.

  > Ejemplo: `fix(ui): corregir desbordamiento de renderizado en cálculo de determinantes`

- `refactor` **(Refactorización):** Reestructuración del código sin alterar su comportamiento externo (vital para prevenir dependencias circulares entre el backend y frontend).

  > Ejemplo: `refactor(math): optimizar cálculo de eigenvalores usando descomposicion QR`

- `docs` **(Documentación):** Alteraciones exclusivas a la documentación técnica o comentarios estructurales.

  > Ejemplo: `docs(api): añadir docstrings a las funciones de transformaciones lineales`

4. Protocolo de Integración (Pull Requests y Code Review)

El proceso de fusión de ramas no es un acto individual; es una decisión de equipo auditable. **El uso del comando `git merge` en terminales locales está absolutamente prohibido.**

1. **Apertura de Pull Request (PR):** Al finalizar un módulo en una rama `feature/`, el autor debe abrir un PR en GitHub apuntando exclusivamente hacia `develop`.
2. **Code Review Cruzado:** Todo PR requiere obligatoriamente la aprobación de al menos un miembro distinto al autor original. El revisor debe verificar rigurosamente que las funciones de Python puro no generen dependencias circulares con los componentes reactivos del frontend.
3. **Fusión (Squash and Merge):** Una vez aprobado, el PR debe integrarse utilizando estrictamente la opción **"Squash and merge"** en la interfaz de GitHub. Esto condensa múltiples commits de desarrollo en un único punto histórico lógico, previniendo la contaminación de la rama `develop`.
4. **Purga de Ramas (Política de Limpieza):** Inmediatamente después de una fusión exitosa, **la rama efímera remota debe ser eliminada obligatoriamente** utilizando el botón "Delete branch" en GitHub. La acumulación de ramas huérfanas en el servidor remoto se considerará una infracción al control de calidad del proyecto MTM0120.